```
import numpy as np
import matplotlib.pyplot as plt
import random

# ==========================================
# 1. DEFINE THE WAREHOUSE ENVIRONMENT
# ==========================================
GRID_SIZE = 12
# Grid cell costs: 1 = Normal Aisle, 5 = Congested Aisle, inf = Obstacle/Shelf
grid_cost = np.ones((GRID_SIZE, GRID_SIZE))

# Place Storage Shelves / Obstacles (Impassable blocks)
obstacles = [
    (2, 2), (2, 3), (2, 4), (2, 7), (2, 8), (2, 9),
    (5, 2), (5, 3), (5, 4), (5, 7), (5, 8), (5, 9),
    (8, 2), (8, 3), (8, 4), (8, 7), (8, 8), (8, 9)
]
for r, c in obstacles:
    grid_cost[r, c] = np.inf

# Place Congested Aisles (High travel time cost)
congested_zones = [
    (4, 5), (5, 5), (6, 5), (7, 5), (3, 6), (4, 6), (5, 6)
]
for r, c in congested_zones:
    grid_cost[r, c] = 5.0

# Define Robot Start and Destination
START = (1, 1)        # Near storage shelf
DESTINATION = (10, 10) # Packing station

# ==========================================
# 2. INITIALIZE ACO PARAMETERS
# ==========================================
NUM_ANTS = 15
ITERATIONS = 40
ALPHA = 1.0       # Pheromone importance
BETA = 2.0        # Heuristic importance (distance/cost priority)
RHO = 0.2         # Evaporation rate
Q = 20.0          # Pheromone deposit factor

# Initialize pheromones matrix for all transitions between cells
# Dimensions: [current_r, current_c, next_r, next_c]
pheromone = np.ones((GRID_SIZE, GRID_SIZE, GRID_SIZE, GRID_SIZE)) * 0.1

def get_neighbors(pos):
    """Returns valid 4-directional adjacent steps within grid bounds."""
    r, c = pos
    neighbors = []
    for dr, dc in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < GRID_SIZE and 0 <= nc < GRID_SIZE:
            if grid_cost[nr, nc] != np.inf:
                neighbors.append((nr, nc))
    return neighbors

def heuristic(pos, dest):
    """Calculates heuristic desirability based on travel cost and distance to destination."""
    r, c = pos
    dist = abs(r - dest[0]) + abs(c - dest[1]) # Manhattan distance
    if dist == 0:
        return 10.0
    # Desirability is inversely proportional to transition cost and remaining distance
    return 1.0 / (grid_cost[r, c] * dist)

# ==========================================
# 3. ACO OPTIMIZATION LOOP
# ==========================================
best_path = []
best_cost = np.inf

for iteration in range(ITERATIONS):
    all_paths = []
    all_costs = []
```

```
        for ant in range(NUM_ANTS):
            current = START
            path = [current]
            visited = set([current])
            stuck = False

            while current != DESTINATION:
                neighbors = get_neighbors(current)
                # Filter out already visited nodes to prevent infinite loops
                allowed = [n for n in neighbors if n not in visited]

                if not allowed:
                    stuck = True
                    break

                # Compute probabilities for choice
                probabilities = []
                for n in allowed:
                    tau = pheromone[current[0], current[1], n[0], n[1]] ** ALPHA
                    eta = heuristic(n, DESTINATION) ** BETA
                    probabilities.append(tau * eta)

                prob_sum = sum(probabilities)
                if prob_sum == 0:
                    probabilities = [1.0 / len(allowed)] * len(allowed)
                else:
                    probabilities = [p / prob_sum for p in probabilities]

                # Choose next cell based on probability distribution
                next_cell = allowed[np.random.choice(len(allowed), p=probabilities)]
                path.append(next_cell)
                visited.add(next_cell)
                current = next_cell

            if not stuck:
                # Calculate total route time/cost cost
                cost = sum(grid_cost[r, c] for r, c in path[1:])
                all_paths.append(path)
                all_costs.append(cost)

                # Track global best solution
                if cost < best_cost:
                    best_cost = cost
                    best_path = path

        # Evaporate pheromones
        pheromone *= (1 - RHO)

        # Deposit pheromones from successful ants
        for path, cost in zip(all_paths, all_costs):
            deposit = Q / cost
            for i in range(len(path) - 1):
                u, v = path[i], path[i+1]
                pheromone[u[0], u[1], v[0], v[1]] += deposit

    print(f"Optimization complete! Best path travel cost/time: {best_cost:.2f}")

    # ==========================================
    # 4. VISUALIZATION
    # ==========================================
    display_grid = np.zeros((GRID_SIZE, GRID_SIZE))
    for r, c in obstacles:
        display_grid[r, c] = -1  # Black for shelves
    for r, c in congested_zones:
        display_grid[r, c] = 0.5 # Gray for traffic congestion

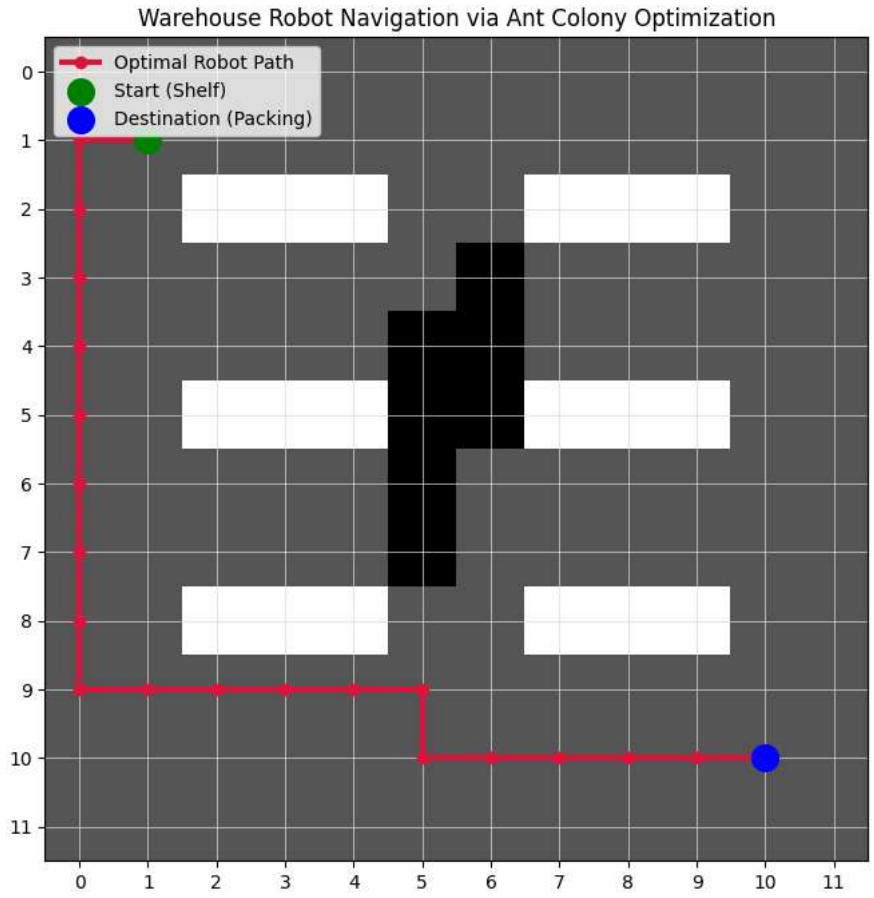
    plt.figure(figsize=(8, 8))
    plt.imshow(display_grid, cmap='binary', origin='upper')

    # Overlay the optimized route
    if best_path:
        path_r, path_c = zip(*best_path)
        plt.plot(path_c, path_r, color='crimson', linewidth=3, marker='o', label='Optimal Robot Path')

    # Plot structural landmarks
    plt.scatter(START[1], START[0], color='green', s=200, zorder=5, label='Start (Shelf)')
    plt.scatter(DESTINATION[1], DESTINATION[0], color='blue', s=200, zorder=5, label='Destination (Packing)')
```

```
plt.grid(True, which='both', color='gainsboro', linestyle='-', linewidth=0.5)
plt.xticks(range(GRID_SIZE))
plt.yticks(range(GRID_SIZE))
plt.title("Warehouse Robot Navigation via Ant Colony Optimization")
plt.legend(loc='upper left')
plt.show()
```

```
Optimization complete! Best path travel cost/time: 20.00
```




```
import numpy as np
import matplotlib.pyplot as plt
import scipy.special as special

# ========================================
# 1. DEFINE PROBLEM AND CONSTANTS
# ========================================
NUM_UNITS = 3               # Number of cooling units
MAX_SERVER_TEMP = 25.0      # Safe upper temperature limit for servers (°C)
HEAT_LOAD = 150.0           # Total heat generated by servers (kW)

# Boundaries for each cooling unit: [Airflow Rate (m³/h), Temp Setting (°C)]
# Airflow bounds: 500 to 3000. Temperature bounds: 16°C to 24°C.
LOWER_BOUNDS = np.array([500, 16] * NUM_UNITS)
UPPER_BOUNDS = np.array([3000, 24] * NUM_UNITS)

# ========================================
# 2. FITNESS FUNCTION (ENERGY & TEMPERATURE)
# ========================================
def calculate_fitness(configuration):
    """Calculates cooling energy and adds a penalty if servers overheat."""
    # Split the 1D configuration array into airflow and temperature lists
    airflows = configuration[0::2]
    temps = configuration[1::2]

    # 1. Calculate Cooling Power Consumption (Simplified physics model)
    # Fan power scales cubically with airflow, compressor power increases for lower temps
    fan_power = sum(0.000002 * (f ** 3) for f in airflows)
    compressor_power = sum(0.5 * (24.0 - t) * (f / 1000.0) for f, t in zip(airflows, temps))
    total_energy = fan_power + compressor_power
```

```
    # 2. Estimate Resulting Server Temperature
    # Higher airflow and lower cooling temperatures decrease server temperature
    total_cooling_capacity = sum(0.0012 * f * (30.0 - t) for f, t in zip(airflows, temps))
    if total_cooling_capacity == 0:
        predicted_server_temp = 100.0
    else:
        predicted_server_temp = 20.0 + (HEAT_LOAD / total_cooling_capacity)

    # 3. Apply heavy penalty if server temperature violates safe limits
    penalty = 0.0
    if predicted_server_temp > MAX_SERVER_TEMP:
        penalty = (predicted_server_temp - MAX_SERVER_TEMP) * 5000.0  # Massive penalty cost

    return total_energy + penalty, predicted_server_temp

# ==========================================
# 3. LÉVY FLIGHT SIMULATION
# ==========================================
def levy_flight(Lambda=1.5):
    """Generates random step sizes following a Lévy distribution."""
    sigma = (special.gamma(1 + Lambda) * np.sin(np.pi * Lambda / 2) /
             (special.gamma((1 + Lambda) / 2) * Lambda * (2 ** ((Lambda - 1) / 2)))) ** (1 / Lambda)
    u = np.random.normal(0, sigma, size=NUM_UNITS * 2)
    v = np.random.normal(0, 1, size=NUM_UNITS * 2)
    step = u / (np.abs(v) ** (1 / Lambda))
    return step

# ==========================================
# 4. CUCKOO SEARCH ALGORITHM LOOP
# ==========================================
POP_SIZE = 20      # Number of nests (population size)
MAX_ITER = 50       # Max generations
Pa = 0.25           # Probability of discovering a bad egg

# Initialize population randomly within bounds
nests = np.zeros((POP_SIZE, NUM_UNITS * 2))
fitness = np.zeros(POP_SIZE)
server_temps = np.zeros(POP_SIZE)

for i in range(POP_SIZE):
    nests[i] = LOWER_BOUNDS + np.random.rand(NUM_UNITS * 2) * (UPPER_BOUNDS - LOWER_BOUNDS)
    fitness[i], server_temps[i] = calculate_fitness(nests[i])

# Track progress for visualization
history_best_energy = []

for iteration in range(MAX_ITER):
    # Step A: Choose a random cuckoo nest and generate a new one via Lévy flight
    cuckoo_idx = np.random.randint(0, POP_SIZE)
    step_size = levy_flight()
    new_nest = nests[cuckoo_idx] + 0.01 * step_size * (nests[cuckoo_idx] - nests[np.random.randint(0, POP_SIZE)])
    new_nest = np.clip(new_nest, LOWER_BOUNDS, UPPER_BOUNDS) # Keep within bounds

    new_fit, new_temp = calculate_fitness(new_nest)

    # Step B: Select a random host nest to replace if the new egg is better
    host_idx = np.random.randint(0, POP_SIZE)
    if new_fit < fitness[host_idx]:
        nests[host_idx] = new_nest
        fitness[host_idx] = new_fit
        server_temps[host_idx] = new_temp

    # Step C: Abandon a fraction (Pa) of the worst nests
    worst_indices = np.argsort(fitness)[-int(Pa * POP_SIZE):]
    for idx in worst_indices:
        nests[idx] = LOWER_BOUNDS + np.random.rand(NUM_UNITS * 2) * (UPPER_BOUNDS - LOWER_BOUNDS)
        fitness[idx], server_temps[idx] = calculate_fitness(nests[idx])

    # Track the absolute best configuration found so far
    best_idx = np.argmin(fitness)
    history_best_energy.append(fitness[best_idx])

# ==========================================
# 5. OUTPUT AND VISUALIZATION
# ==========================================
best_idx = np.argmin(fitness)
```

```
best_config = nests[best_idx]
best_energy, final_server_temp = calculate_fitness(best_config)

print("="*45)
print("  OPTIMIZATION COMPLETE (CUCKOO SEARCH)   ")
print("="*45)
print(f"Minimized Cooling Energy Power: {best_energy:.2f} kW")
print(f"Final Server Temperature: {final_server_temp:.2f}°C (Safe Max: {MAX_SERVER_TEMP}°C)")
print("-"*45)
for u in range(NUM_UNITS):
    print(f"Cooling Unit {u+1} -> Airflow: {best_config[u*2]:.1f} m³/h | Temp Setpoint: {best_config[u*2+1]:.1f}°C")
print("="*45)

# Plot Optimization Convergence Chart
plt.figure(figsize=(8, 4))
plt.plot(history_best_energy, color='dodgerblue', linewidth=2.5, marker='x', markevery=5)
plt.title("Data Center Energy Optimization over Iterations")
plt.xlabel("Iteration / Generation")
plt.ylabel("Total Energy Cost (lower is better)")
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

```
=============================================
  OPTIMIZATION COMPLETE (CUCKOO SEARCH)
=============================================
Minimized Cooling Energy Power: 4411.01 kW
Final Server Temperature: 25.41°C (Safe Max: 25.0°C)
---------------------------------------------
Cooling Unit 1 -> Airflow: 568.6 m³/h | Temp Setpoint: 19.6°C
Cooling Unit 2 -> Airflow: 592.7 m³/h | Temp Setpoint: 20.6°C
Cooling Unit 3 -> Airflow: 919.2 m³/h | Temp Setpoint: 17.4°C
=============================================
```

